

COLLECTION IN JAVA

CHEATSHEET — PART 1

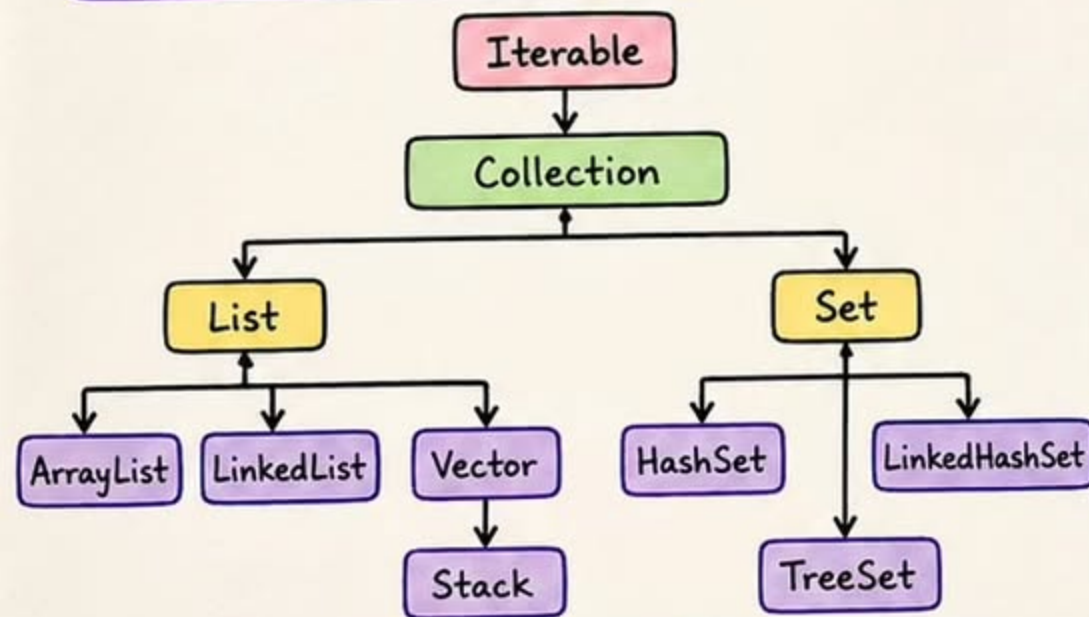
1) WHAT IS COLLECTION ?

Collection in Java is a framework that provides an architecture to **store** and **manipulate** a group of objects.



It helps us to perform operations like **add**, **remove**, **search**, **sort** etc. easily.

3) COLLECTION HIERARCHY



5) IMPORTANT FEATURES

- ✓ Collection can grow dynamically.
- ✓ Different implementations for different needs.
- ✓ Fail-Fast and Fail-Safe behavior.
- ✓ Iterator is used to traverse collection.



2) WHY COLLECTION ?

- ★ Reusability
- ★ High Performance
- ★ Easy to Manipulate
- ★ Reduces Programming Effort
- ★ Standard Data Structure

4) MAIN INTERFACES

- **Iterable** → Root interface
- **Collection** → Subinterface of Iterable
- **List** → Ordered collection allows duplicates
- **Set** → Unordered collection does not allow duplicates
- **Queue** → Interface for queue data structure
- **Map** → Key-Value pair (Covered in Part 2)

6) SIMPLE EXAMPLE

Ajeet Yadav

```
List<String> list = new ArrayList<>();  
list.add("Aadi");  
list.add("Java");  
list.add("Collection");  
System.out.println(list);  
// Output: [Aadi, Java, Collection]
```



Collections make our code **clean**, **efficient** and easy to maintain.

Stay Consistent
Keep Learning ❤️

LIST IN JAVA

CHEATSHEET - PART 1

WHAT IS LIST ?

List is an ordered collection that allows **duplicate** elements.

1) INTRODUCTION

- List is an interface in Java Collection Framework.
- It maintains **insertion order**.
- It allows **duplicate** and **null** elements.

2) LIST IMPLEMENTATIONS

ArrayList	Resizable array. Inserts/Reads are fast.
LinkedList	Doubly linked list. Good for frequent inserts and deletes.
Vector	Similar to ArrayList but synchronized (Thread-safe).
Stack	Extends Vector. LIFO (Last In First Out).

5) VISUALIZATION (ArrayList)

Index →	0	1	2	3	4
Elements →	10	20	30	40	50

`list.add(60)`

Index →	0	1	2	3	4	5
Elements →	10	20	30	40	50	60

`list.remove(2)`

Index →	0	1	2	3	4
Elements →	10	20	40	50	60

Order is maintained always! 😊

REMEMBER

List = Ordered + Duplicates allowed
Choose implementation as per need.

3) DECLARATION & CREATION

```
List<String> list = new ArrayList<>();  
List<Integer> list2 = new LinkedList<>();  
List<String> list3 = new Vector<>();
```

4) COMMON METHODS

<code>add(E e)</code>	→ Adds element at the end
<code>add(int index, E e)</code>	→ Adds element at given index
<code>get(int index)</code>	→ Returns element at index
<code>set(int index, E e)</code>	→ Replaces element at index
<code>remove(int index)</code>	→ Removes element at index
<code>remove(Object o)</code>	→ Removes first occurrence
<code>size()</code>	→ Returns number of elements
<code>isEmpty()</code>	→ Checks if list is empty
<code>contains(Object o)</code>	→ Checks if element exists

6) SIMPLE EXAMPLE

```
import java.util.*;  
public class ListExample {  
    public static void main(String[] args) {  
        List<String> list = new ArrayList<>();  
        list.add("Aadi");  
        list.add("Java");  
        list.add("List");  
        System.out.println(list); // [Aadi, Java, List]  
    }  
}
```

Ajeet Yadav

→ TIP



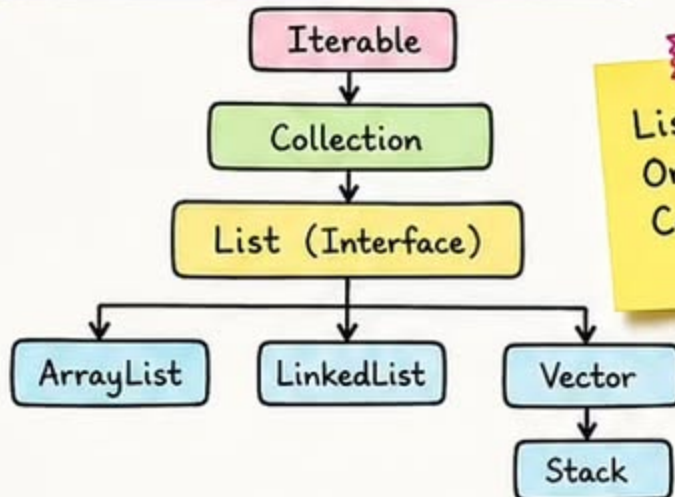
Use ArrayList for fast access.
Use LinkedList for fast insert/delete.

KEEP PRACTICING
KEEP LEARNING

LIST IN JAVA

CHEATSHEET – PART 2

1) LIST INTERFACE HIERARCHY



List is an
Ordered
Collection

3) COMMON METHODS OF LIST

<code>add(E e)</code>	→ Adds element at the end
<code>add(int index, E e)</code>	→ Adds element at given index
<code>get(int index)</code>	→ Returns element at index
<code>set(int index, E e)</code>	→ Replaces element at index
<code>remove(int index)</code>	→ Removes element at index
<code>remove(Object o)</code>	→ Removes first occurrence of element
<code>size()</code>	→ Returns number of elements
<code>isEmpty()</code>	→ Checks if list is empty
<code>contains(Object o)</code>	→ Checks if element exists
<code>indexOf(Object o)</code>	→ Returns index of first occurrence
<code>lastIndexOf(Object o)</code>	→ Returns index of last occurrence

5) COMPARISON OF LIST IMPLEMENTATIONS

Implementation	Underlying Data Structure	Insertion/Deletion	Access	Thread Safe
ArrayList	Resizable Array	Slow (middle) Fast (end)	Fast (O(1))	No
LinkedList	Doubly Linked List	Fast	Slow (O(n))	No
Vector	Resizable Array	Slow	Fast	Yes
Stack	Extends Vector	Fast (push/pop)	Fast	Yes

2) KEY POINTS ABOUT LIST

- ✓ Maintains **insertion order**.
- ✓ Allows **duplicate** elements.
- ✓ Allows **null** elements.
- ✓ Index starts from **0**.
- ✓ Elements can be accessed using index.

List Implementations

- ArrayList – Resizable array
- LinkedList – Doubly linked list
- Vector – Legacy class (Thread-safe)
- Stack – Extends Vector

4) CODE EXAMPLES

```
import java.util.*;
public class ListExample {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>();
        list.add("Aadi"); // [Aadi]
        list.add("Java"); // [Aadi, Java]
        list.add("List"); // [Aadi, Java, List]
        list.add(1, "is"); // [Aadi, is, Java, List]
        System.out.println(list.get(2)); // Java
        list.set(2, "Collection");
        System.out.println(list);
        list.remove(1); // [Aadi, Collection, List]
        System.out.println(list.size()); // 3
        System.out.println(list.contains("Java")); // true
    }
}
```

Practice
Makes
Perfect!

Ajeet Yadav

6) QUICK SUMMARY

- ★ Use ArrayList for general purpose.
- ★ Use LinkedList when frequent insertions/deletions in middle.
- ★ Use Vector/Stack only when thread-safety is required (legacy).



Keep Practicing
Keep Learning

Remember : List = Ordered + Duplicates Allowed + Indexed Access

SET IN JAVA

CHEATSHEET - PART 1

1) WHAT IS SET ?

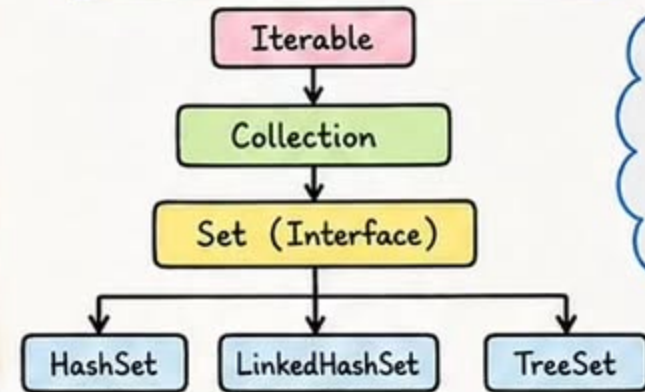
- Set is an interface in Java Collection Framework.
- It represents an **unordered** collection of **unique** elements.
- It does not allow **duplicate** elements.
- It allows only one **null** element.

UNIQUE

Remember

Set = No Duplicates
Order not guaranteed
(depends on implementation)

2) SET INTERFACE HIERARCHY



Set contains only **unique** elements

3) IMPLEMENTING CLASSES OF SET

Class	Description
HashSet	Implemented using Hash Table . No order is maintained. Allows one null value.
LinkedHashSet	Maintains insertion order . Implemented using Hash Table + Linked List. Allows one null value.
TreeSet	Elements are sorted in natural ascending order. Implemented using Red-Black Tree. Does not allow null value.

4) COMMON METHODS OF SET (from Set<E>)

`add(E e)` → Adds element. Returns true if added.
`addAll(Collection<? extends E> c)` → Adds all elements.
`remove(Object o)` → Removes element. Returns true if removed.
`contains(Object o)` → Checks if element exists.
`size()` → Returns number of elements.
`isEmpty()` → Checks if set is empty.
`clear()` → Removes all elements.
`iterator()` → Returns an iterator.
`toArray()` → Returns array of elements.
`containsAll(Collection<?> c)` → Checks if all elements exists.

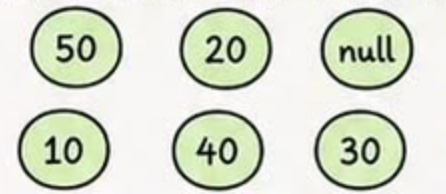
5) EXAMPLE

```
import java.util.*;
public class SetExample {
    public static void main(String[] args) {
        Set<String> set = new HashSet<>();
        set.add("Apple");
        set.add("Banana");
        set.add("Apple"); // Duplicate - will not be added
        set.add(null);
        set.add(null); // Only one null allowed
        System.out.println(set); // Output may vary
        System.out.println("Size: " + set.size());
    }
}
```

Duplicate elements are ignored

6) VISUALIZATION (HashSet)

No Order is maintained



One null element allowed

7) QUICK SUMMARY

- ✓ Set stores unique elements.
- ✓ Duplicate elements are not allowed.
- ✓ Only one null value is allowed (except TreeSet).
- ✓ Order is not guaranteed (except LinkedHashSet & TreeSet).
- ✓ Use Set when you only care about uniqueness, not order.

Ajeet Yadav

TIP

Use HashSet for best performance.
Use LinkedHashSet if you need insertion order.
Use TreeSet if you need sorted elements.

SET = UNIQUE + (UNORDERED / SORTED)

SET IN JAVA

CHEATSHEET – PART 2

Remember

- Set = No Duplicates
- Only one null allowed
- Order is not guaranteed

1) SET OPERATIONS

Operation	Method	Description
Union	<code>addAll()</code>	Combines two sets
Intersection	<code>retainAll()</code>	Keeps common elements
Difference	<code>removeAll()</code>	Removes elements present in another set
Symmetric Difference	$(A \cup B) - (A \cap B)$	Elements in either A or B but not in both

Example:

Set A = {1, 2, 3, 4}

Set B = {3, 4, 5, 6}

Union $(A \cup B) = \{1, 2, 3, 4, 5, 6\}$

Intersection $(A \cap B) = \{3, 4\}$

Difference $(A - B) = \{1, 2\}$

Symmetric Diff = {1, 2, 5, 6}

3) SUBSET & SUPERSET CHECK

- A set A is **subset** of B if every element of A is in B.
- A set A is **superset** of B if every element of B is in A.

A is Subset of B	<code>A.containsAll(B)</code>
A is Superset of B	<code>B.containsAll(A)</code>

Fast way to compare sets!

4) EXAMPLE – SUBSET & SUPERSET

Set A = {1, 2}

Set B = {1, 2, 3, 4}

- A is subset of B → `A.containsAll(B) = true`
- A is superset of B → `B.containsAll(A) = false`
- B is superset of A → `B.containsAll(A) = true`

Use `containsAll()` is the key!

6) PERFORMANCE (Time Complexity)

Operation	HashSet	LinkedHashSet	TreeSet
<code>add(E e)</code>	O(1) avg.	O(1) avg.	O(log n)
<code>remove(Object o)</code>	O(1) avg.	O(1) avg.	O(log n)
<code>contains(Object o)</code>	O(1) avg.	O(1) avg.	O(log n)

HashSet is fastest for most operations!



2) CODE EXAMPLE – SET OPERATIONS

```
import java.util.*;
public class SetOperations {
    public static void main(String[] args) {
        Set<Integer> A = new HashSet<>(Arrays.asList(1,2,3,4));
        Set<Integer> B = new HashSet<>(Arrays.asList(3,4,5,6));

        // Union
        Set<Integer> union = new HashSet<>(A);
        union.addAll(B); // [1, 2, 3, 4, 5, 6]

        // Intersection
        Set<Integer> intersection = new HashSet<>(A);
        intersection.retainAll(B); // [3, 4]

        // Difference A - B
        Set<Integer> diff = new HashSet<>(A);
        diff.removeAll(B); // [1, 2]

        // Symmetric Difference
        Set<Integer> symDiff = new HashSet<>(A);
        symDiff.addAll(B);
        Set<Integer> temp = new HashSet<>(A);
        temp.retainAll(B);
        symDiff.removeAll(temp); // [1, 2, 5, 6]
    }
}
```

Set Operations Made Easy! 😊

5) SET TO OTHER COLLECTIONS

Conversion	Example
Set to List	<code>List<Integer> list = new ArrayList<>(set);</code>
Set to Array	<code>Integer[] arr = set.toArray(new Integer[0]);</code>
Set to String	<code>String str = set.toString(); // [1, 2, 3]</code>

7) QUICK SUMMARY

- ✓ Set stores only unique elements.
- ✓ Does not maintain insertion order (except LinkedHashSet).
- ✓ Only one null element is allowed.
- ✓ Use HashSet for best performance.
- ✓ Use LinkedHashSet when insertion order matters.
- ✓ Use TreeSet when sorted order is required.

Ajeet Yadav

TIP: Choose the right Set implementation based on your need!

Speed → HashSet | Order → LinkedHashSet | Sorted → TreeSet

Practice More, Code Better! 😊

Map

stores data in
Key-Value
pairs.

MAP in JAVA

~ CHEATSHEET (Part 1) ~

Key Points

- Keys are Unique
- Values can be Duplicate
- Order depends on implementation

1. What is Map?

Map is an **Interface** in Java of `java.util` package. It contains **Key-Value** pairs.

Like
Dictionary

2. Map Interface Methods

• put(key, value)	→ Insert or Update
• get(key)	→ Get value by key
• remove(key)	→ Remove key-value pair
• containsKey(key)	→ Check key exists
• containsValue(value)	→ Check value exists
• size()	→ Return number of pairs
• isEmpty()	→ Check map is empty
• keySet()	→ Return all keys
• values()	→ Return all values
• entrySet()	→ Return key-value pairs

3. Types of Map

- **HashMap**
(Unordered, Not Sorted)
- **LinkedHashMap**
(Maintains Insertion Order)
- **TreeMap**
(Sorted by Keys)
- **Hashtable**
(Synchronized, Legacy)
- **ConcurrentHashMap**
(Thread-Safe)

4. Example

```
import java.util.*;
public class MapExample {
    public static void main(String[] args) {
        Map<Integer, String> map = new HashMap<>();

        map.put(1, "Aadi");
        map.put(2, "Pooja");
        map.put(3, "Laxman");

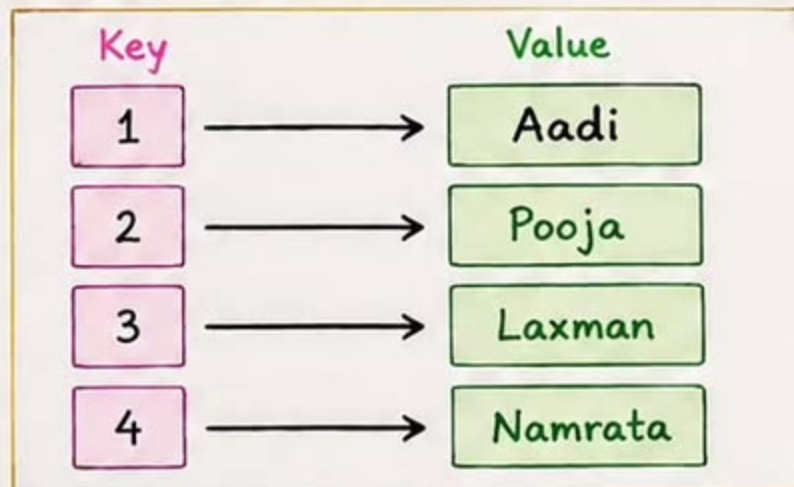
        System.out.println("Map: " + map);
        System.out.println("Value for key 2: " + map.get(2));
        System.out.println("Contains key 3? " +
            map.containsKey(3));

        map.remove(2);
        System.out.println("After remove: " + map);
    }
}
```

// Output

Map: {1=Aadi, 2=Pooja, 3=Laxman}
Value for key 2: Pooja
Contains key 3? true
After remove: {1=Aadi, 3=Laxman}

5. Visualization



Example: {1=Aadi, 2=Pooja, 3=Laxman, 4=Namrata}

Every Key
is Unique
but Values
can repeat.

6. Quick Notes

- Use HashMap for general purpose.
- Use LinkedHashMap when order is important.
- Use TreeMap when you need sorted keys.
- Use ConcurrentHashMap for multi-threading.

Remember

Map does not
allow duplicate
keys.

Ajeet Yadav

Map Reminder

Map stores data in Key-Value pairs.
Keys are unique.
Values can repeat.

MAP in JAVA

CHEATSHEET (Part 2)

Why Map?

- Fast search by key
- No duplicate keys
- Very useful in real world applications

1. Common Operations

- **put(key, value)**
→ Insert or Update
- **get(key)**
→ Get value by key
- **remove(key)**
→ Remove key-value pair
- **containsKey(key)**
→ Check key exists
- **containsValue(value)**
→ Check value exists
- **replace(key, value)**
→ Replace value for key
- **putIfAbsent(key, value)**
→ Put only if key not exists
- **clear()** → Remove all entries
- **size()** → Return number of entries
- **isEmpty()** → Check map is empty

2. Iterating Map

A) Using entrySet() - Best Way

```
for (Map.Entry<K,V> e : map.entrySet()) {  
    System.out.println(e.getKey()+"->" + e.getValue());  
}
```

B) Using keySet()

```
for (K key : map.keySet()) {  
    System.out.println(key + " -> " + map.get(key));  
}
```

C) Using values()

```
for (V value : map.values()) {  
    System.out.println(value);  
}
```

★ **entrySet()** is most efficient when you need both key & value.

3. Map Implementation Classes

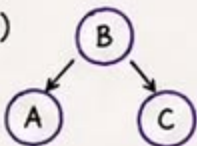
- **HashMap<K,V>**
→ Unordered, Not Sorted
→ Allows null key & multiple null values
→ Fast performance



- **LinkedHashMap<K,V>**
→ Maintains Insertion Order
→ Slightly slower than HashMap



- **TreeMap<K,V>**
→ Sorted by Keys (Ascending Order)
→ Does not allow null key
→ Slower than HashMap



- **Hashtable<K,V>**
→ Synchronized (Thread-Safe)
→ Legacy class
→ Does not allow null key or values

4. More Examples

```
Map<String, Integer> map = new HashMap<>();  
map.put("One", 1);  
map.put("Two", 2);  
map.put("Three", 3);  
map.putIfAbsent("Two", 22); // will not update  
map.replace("Three", 33); // value replaced  
  
System.out.println(map.get("Two")); // 2  
System.out.println(map.containsKey("One")); // true  
System.out.println(map.containsValue(33)); // true  
map.remove("One");  
System.out.println(map); // {Two=2, Three=33}
```

5. Mini Visualization (HashMap)

Key	Hash	Bucket
Aadi	23	(Aadi, 101)
Pooja	15	(Pooja, 102)
Laxman	23	(Laxman, 103)
Om	23	(Om, 104)

★ Same hash values can go to same bucket.
Java handles it using Linked List (Chaining).

6. Tips

Ajeet Yadav

- 💡 Use HashMap for general purpose.
- 💡 Use LinkedHashMap when order matters.
- 💡 Use TreeMap when you need sorted keys.
- 💡 Do not modify map while iterating (use iterator remove if needed).

Remember

Map does not allow duplicate keys!

Summary

Maps are powerful in Java for storing and retrieving data using unique keys. Choose the right implementation based on your requirement.

Practice

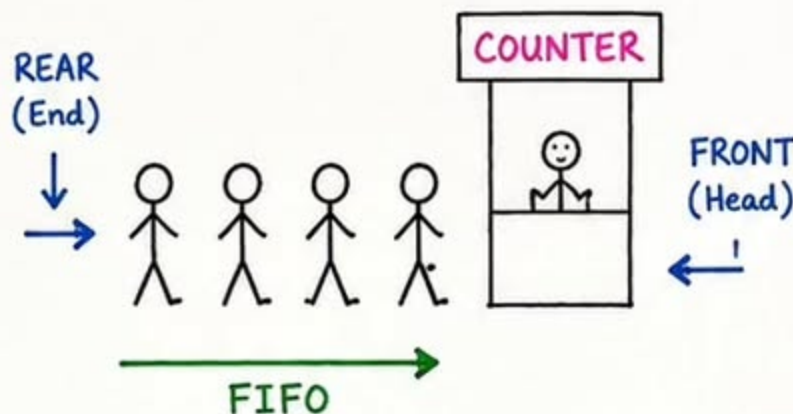
Code, Test, Debug and become a Java Master!

Queue in Java

CHEATSHEET Part 1

1. What is Queue?

- ★ Queue is a collection of elements that follows FIFO order.
- ★ FIFO = First In, First Out
- ★ The element added first will be removed first.
- ★ Like a real-life queue (Line).



Key Point

- Add from Rear
- Remove from Front
- FIFO Order

2. Queue Operations

Method	Description
<code>add(e)</code>	Inserts element <code>e</code> at rear. Throws <u>exception</u> if full.
<code>offer(e)</code>	Inserts element <code>e</code> at rear. Returns <u>false</u> if full.
<code>remove()</code>	Removes & returns front element. Throws <u>exception</u> if empty.
<code>poll()</code>	Removes & returns front element. Returns <u>null</u> if empty.
<code>element()</code>	Returns front element. Throws exception if empty.
<code>peek()</code>	Returns front element. Returns <u>null</u> if empty.

3. Types of Queue in Java

- ★ `LinkedList` (Most used)
- ★ `PriorityQueue` (Sorted by priority)
- ★ `ArrayDeque` (Resizing Array)
- ★ `BlockingQueue` (Used in Multithreading)

Ajeet Yadav

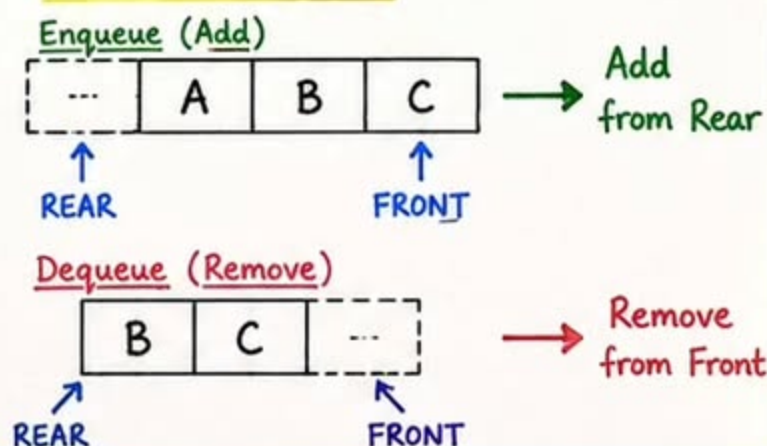
Where to Use?

- ✓ CPU Scheduling
- ✓ Print Queue
- ✓ BFS (Graph Traversal)
- ✓ Handling Requests in Servers

4. Example using LinkedList

```
Queue<String> q = new LinkedList<>();
q.offer("A");
q.offer("B");
q.offer("C");
System.out.println(q.peek()); // A
System.out.println(q.poll()); // A
System.out.println(q.poll()); // B
System.out.println(q);        // [C]
```

5. Visualization



Memory Trick

F I F O

First In
First Out



Remember: Queue = **FIFO** (First In, First Out)



Queue in Java

CHEATSHEET Part 2

1. Queue Interface

- Queue is an Interface in java.util package.
- It extends Collection interface.

Declaration

```
public interface Queue<E>
    extends Collection<E> {
        // methods
    }
```

Types of Queue

- ★ LinkedList (most used)
- ★ PriorityQueue
- ★ ArrayDeque
- ★ BlockingQueue (for multithreading) 😊

2. Common Methods

Method	Description
add(e)	Inserts element, throws exception if full.
offer(e)	Inserts element, returns <u>false</u> if full.
remove()	Removes head, throws exception if empty.
poll()	Removes head, returns <u>null</u> if empty.
element()	Returns head, throws exception if empty.
peek()	Returns head, returns <u>null</u> if empty.

Key Points

- ✓ Follows FIFO order
- ✓ Insert from Rear
- ✓ Remove from Front
- ✓ Queue can be bounded or unbounded 😊

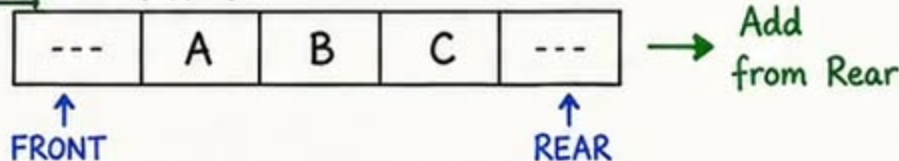
3. Example Using LinkedList

```
Queue<String> q = new LinkedList<>();
q.offer("A"); // [A]
q.offer("B"); // [A, B]
q.offer("C"); // [A, B, C]
System.out.println(q.peek()); // A
System.out.println(q.poll()); // A removed
System.out.println(q); // [B, C]
```

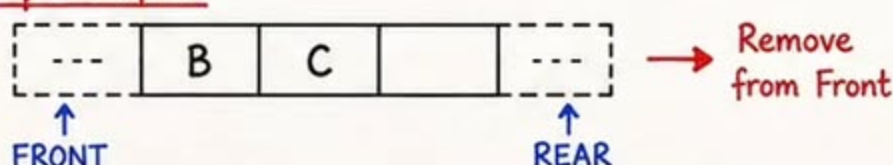
💡 offer() is preferred over add() because it doesn't throw exception.

4. Queue Visualization

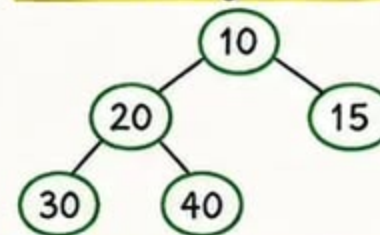
Enqueue (offer)



Dequeue (poll)

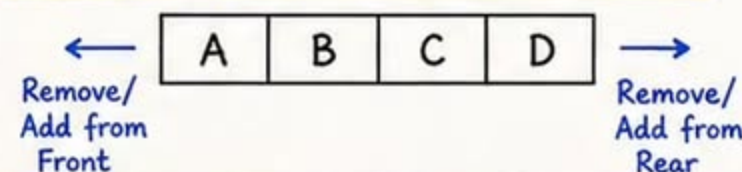


5. PriorityQueue (Min-Heap)



Elements are ordered by priority (natural order).

6. ArrayDeque (Better than Stack & Queue)



ArrayDeque supports both Queue and Deque operations.

★ Remember: Queue = **FIFO** (First In, First Out) ★

Ajeet Yadav